语音播报颜色

语音播报颜色
- 实验简介
- 实验效果
- 代码解析

实验简介

这一节我们来学习一下使用K230实现颜色识别并通过喇叭播放颜色的功能

本节例程代码位于：【 源码汇总 / 10.Media/07.voice_color.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

我们在这里设置播放的语音文件路径，本例程可以识别红色、绿色、蓝色。

```
wav_files = ["/sdcard/audio/detect_red.wav", "/sdcard/audio/detect_green.wav",
"/sdcard/audio/detect_blue.wav"]
```

如果需要生成自定义语音，请在上一节课【文字转语音】中修改文字并生成语音文件，再将语音文件路径添加到wav_files中。

实验效果

点击CanMV IDE左下角的运行按钮，K230会开启摄像头，并且识别当前画面中的颜色，如果识别到红色，即可播报'识别完成，这是红色'，如果识别到绿色，即可播报'识别完成，这是绿色'，如果识别到蓝色，即可播报'识别完成，这是蓝色'。

代码解析

颜色识别处理函数，主要功能是根据颜色的LAB值来区分图像中的颜色，识别并框出符合当前lab值的颜色块。

```
def process_blobs(img, threshold_idx):
    """处理颜色区块检测 / Process color blob detection"""
    area_threshold = 5000
    blobs = img.find_blobs([COLOR_THRESHOLDS[threshold_idx]],
area_threshold=area_threshold, merge=True)
    max_area = -1
    if blobs:
        for blob in blobs:
            if blob.area() > max_area:
                max_area = blob.area()
            # 绘制检测框和标记 / Draw detection box and markers
```

```
            img.draw_rectangle(blob[0:4], thickness=4,
color=DRAW_COLORS[threshold_idx])
            img.draw_cross(blob[5], blob[6], thickness=2)
            img.draw_string_advanced(blob[0], blob[1]-35, 30,
                                     COLOR_LABELS[threshold_idx],
                                     color=DRAW_COLORS[threshold_idx])
    return max_area
```

颜色对应的lab阈值可以在COLOR_THRESHOLDS变量中修改或增加

```
COLOR_THRESHOLDS = [
    (0, 66, 7, 127, 3, 127),    # 红色阈值 / Red threshold
    (42, 100, -128, -17, 6, 66),     # 绿色阈值 / Green threshold
    (43, 99, -43, -4, -56, -7),       # 蓝色阈值 / Blue threshold
]
```

多次循环检测图像，返回的面积哪个最大，就表示识别到当前是什么颜色。

```
for i in range(3):
    value = process_blobs(img, i)
    if value > max_value[1]:
        max_value[1] = value
        max_value[0] = i
        if max_value[0] != -1:
            if last_color == max_value[0] and play_flag == -1:
                color_count = color_count + 1
                if color_count > color_igonre:
                    print("color:", max_value[0], max_value[1])
                    play_flag = max_value[0]
                    color_count = 0
```

然后通过新建一个后台线程来播放识别到的颜色。

```
_thread.start_new_thread(func,("THREAD_1",))

def func(name):
    time.sleep(.1)
    global play_flag, wav_files

    while True:
        os.exitpoint()
        if play_flag != -1:
            play_audio(wav_files[play_flag])
            play_flag = -1
        time.sleep(.01)
```

具体播放语音音频文件的函数是play_audio，主要功能是读取传入的wav文件路径，并驱动扬声器播放声音。

```
def play_audio(wav_file):
    global p
    spk.enable()  # 启用扬声器 Enable speaker
    # 有关音频流的宏变量
    SAMPLE_RATE = 24000          # 采样率24000Hz,即每秒采样24000次
```

```
    CHANNELS = 1                # 通道数 1为单声道，2为立体声
    FORMAT = paInt16            # 音频输入输出格式 paInt16
    CHUNK = int(24000*0.3)     # 每次读取音频数据的帧数，设置为0.3s的帧数24000*0.3=7200

    # 用于播放音频
    output_stream =
p.open(format=FORMAT,channels=CHANNELS,rate=SAMPLE_RATE,output=True,frames_per_b
uffer=CHUNK)
    wf = wave.open(wav_file, "rb")
    wav_data = wf.read_frames(CHUNK)
    while wav_data:
        output_stream.write(wav_data)
        wav_data = wf.read_frames(CHUNK)
    time.sleep(2) # 时间缓冲，用于播放声音
    wf.close()
    output_stream.stop_stream()
    output_stream.close()
    spk.disable()  # 禁用扬声器 Disable speaker
```